РО 1.3. Базовое администрирование Linux

Лекция 11.
Файлы, каталоги, ссылки и структура файловой системы Linux

Список учебных вопросов

1. Единое дерево каталогов Linux и отличие от букв дисков Windows
2. Назначение основных каталогов: /, /home, /etc, /var, /usr, /bin, /sbin, /tmp, /root
3. Файл и каталог в Linux
4. Типы файлов: обычный файл, каталог, символическая ссылка, устройство, сокет, pipe
5. Базовые операции: mkdir, touch, cp, mv, rm, rmdir
6. Поиск файлов: find, locate на уровне назначения
7. inode: назначение и связь с именем файла
8. Жесткие ссылки и символические ссылки
9. Типовые ошибки при удалении, копировании и работе со ссылками

1. Единое дерево каталогов Linux и отличие от букв дисков Windows

Файловая система — способ организации хранения файлов, каталогов и служебных объектов.

В Linux все начинается с корня /, даже если в системе несколько дисков.

Диск или раздел подключается в дерево каталогов через точку монтирования.

Администратор мыслит не буквами дисков, а путями и точками монтирования.

Почему это важно администратору

- Конфигурация, журналы, программы и данные пользователей лежат в разных частях дерева;
- Ошибка в пути может изменить не тот файл или удалить не тот каталог;
- При нехватке места важно понимать, какая файловая система заполнена;
- Будущие серверные службы требуют уверенной ориентации в структуре Linux;

ЕДИНОЕ ДЕРЕВО КАТАЛОГОВ LINUX

Вся файловая система начинается с корня: / (root)

Linux использует единое дерево каталогов (FHS – Filesystem Hierarchy Standard).
Все файлы и устройства находятся в единой структуре,
начиная с корневого каталога /

КОРЕНЬ ФАЙЛОВОЙ СИСТЕМЫ

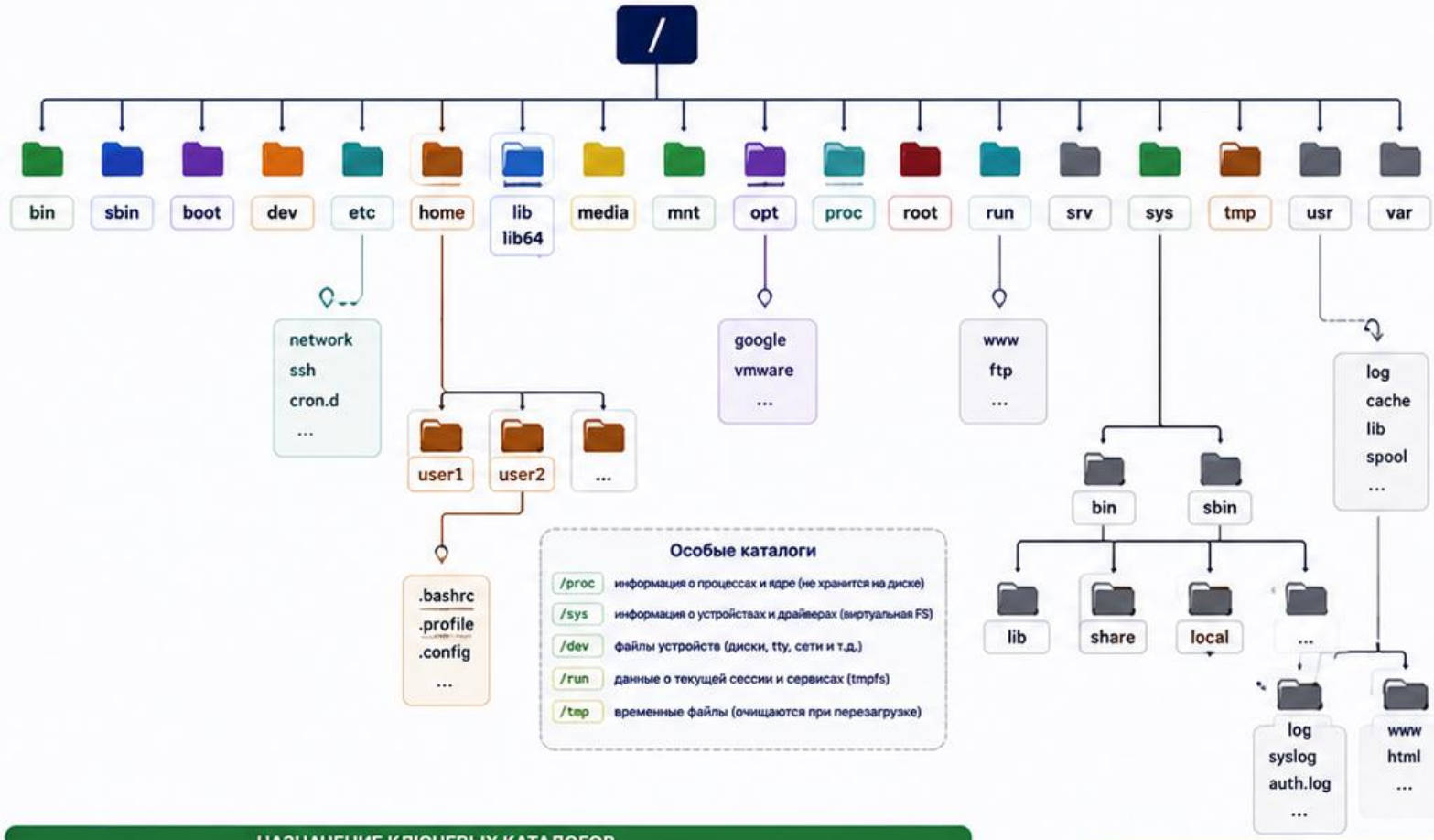
/

Корневой каталог.
Начало всего дерева.
Содержит стандартные каталоги системы.

ОСНОВНЫЕ КАТАЛОГИ

/bin	Основные пользовательские команды (ls, cp, mv, cat и т.д.)
/sbin	Системные команды администрирования
/boot	Файлы загрузчика и ядра Linux
/dev	Файлы устройств
/etc	Конфигурационные файлы системы и приложений
/home	Домашние каталоги пользователей
/lib, /lib64	Библиотеки для программ из /bin и /sbin
/media	Точки монтирования съёмных устройств
/mnt	Временные точки монтирования
/opt	Дополнительные программы (стороннее ПО)
/proc	Виртуальная файловая система с информацией о процессах и системе
/root	Домашний каталог пользователя root
/run	Данные о работающей системе (PID-файлы, сокеты и т.д.)

ДЕРЕВО КАТАЛОГОВ



Особые каталоги

/proc	информация о процессах и ядре (не хранится на диске)
/sys	информация о устройствах и драйверах (виртуальная FS)
/dev	файлы устройств (диски, tty, сети и т.д.)
/run	данные о текущей сессии и сервисах (tmpfs)
/tmp	временные файлы (очищаются при перезагрузке)

ЧТО НУЖНО ЗНАТЬ

Всё является файлом
В Linux всё: файлы, каталоги, устройства, сокеты – представлено в виде файлов.

Пути бывают двух типов
- Абсолютный: начинается с / (например, /etc/hosts)
- Относительный: от текущего каталога (например, ../docs)

Доступ к файлам
Права доступа определяют, кто может читать, писать и выполнять файлы/каталоги.

Разделы и файловые системы
Каталоги могут располагаться на разных разделах и файловых системах, смонтированных в дерево.

Команды для навигации
`ls` – просмотр содержимого
`cd` – переход между каталогами
`pwd` – показать текущий каталог

ПРИМЕРЫ МОНТИРОВАНИЯ

/	корневой раздел (ext4)
/home	отдельный раздел (ext4)
/var	отдельный раздел (xfs)
/mnt/usb	флешка или диск (vfat, ntfs)
/srv/www	данные веб-сайта

ПРИМЕРЫ ПОЛНЫХ ПУТЕЙ

- /etc/hosts
- /home/user/docs/file.txt
- /var/log/syslog
- /usr/local/bin/script.sh
- /mnt/usb/data/report.pdf

НАЗНАЧЕНИЕ КЛЮЧЕВЫХ КАТАЛОГОВ

/bin, /sbin	важные системные команды	/usr	пользовательские программы и данные
/etc	настройки всей системы	/opt	дополнительное ПО от сторонних разработчиков
/home	личные данные пользователей	/boot	файлы для загрузки системы
/var	изменяемые данные (логи, кэш, почта и др.)	/dev, /proc, /sys	информация о системе и устройствах

ПОЛЕЗНЫЕ КОМАНДЫ

```
ls /                    # список корневого каталога
tree /                  # дерево каталогов (нужна утилита tree)
df -h                   # использование дискового пространства
mount                   # список смонтированных файловых систем
find / -name file.txt   # поиск файла в системе
```

Понимание структуры каталогов — основа эффективной работы в Linux!

Практика: исследуйте систему с помощью `ls`, `cd` и `tree`.

Просмотр корня файловой системы

Задача: увидеть верхний уровень дерева Linux
Команда: **ls /**
Пример результата: **bin boot dev etc home root usr var**
…
Это не диски, а основные ветви единого дерева каталогов

2. Назначение основных каталогов Linux

- Корень **/** — начало всей файловой системы
- **/home** хранит домашние каталоги обычных пользователей
- **/etc** хранит системную конфигурацию
- **/var** хранит изменяемые данные: журналы, кэш, очереди
- **/usr** хранит программы, библиотеки и общие ресурсы

Системные и пользовательские зоны

- **/root** — домашний каталог администратора root, не то же самое, что /
- **/tmp** — временные файлы, доступные разным программам и пользователям
- **/bin** и **/sbin** содержат базовые исполняемые файлы

Разделение каталогов помогает сопровождать систему и искать проблемы быстрее.

ОСНОВНЫЕ КАТАЛОГИ LINUX

Ключевые директории единого дерева файловой системы

В Linux всё является файлом: обычные файлы, каталоги, устройства, сокеты, каналы (pipe) и т.д.

Все пути начинаются с корня: / (root)

СИСТЕМНЫЕ КАТАЛОГИ

/ (root) — Корневой каталог
Начало всего дерева файловой системы.
Примеры: `/bin /etc /home /usr`

> — Основные команды
Критически важные команды для всех пользователей. Должны работать в однопользовательском режиме.
Примеры: `ls cp mv cat bash`

/sbin — Системные команды
Системные утилиты для администрирования. Обычно используются с правами root.
Примеры: `fdisk reboot ip ifconfig`

/etc — Конфигурационные файлы
Настройки системы и приложений. Хранит конфигурацию в текстовом виде.
Примеры: `hosts fstab ssh nginx`

/home — Домашние каталоги пользователей
Каждый пользователь имеет свой каталог для личных файлов и настроек.
Примеры: `/home/user/Documents`

/root — Домашний каталог суперпользователя
Личные файлы и настройки пользователя root.
Примеры: `/root/.bashrc`

/usr — Пользовательские программы и данные
Большинство программ, библиотек, документации и общих данных (не зависят от системы).
Подкаталоги: `bin lib share local`

/var — Изменяемые данные
Файлы, которые часто изменяются в процессе работы системы и сервисов.
Примеры: `log cache spool tmp`

/tmp — Временные файлы
Временные файлы, которые могут быть удалены при перезагрузке системы.
Примеры: `/tmp/somefile`

/boot — Файлы загрузки системы
Ядро, initramfs и загрузчик (GRUB). Необходим для старта системы.
Примеры: `vmlinuz grub.cfg`

/dev — Файлы устройств
Представление аппаратных устройств в виде файлов.
Примеры: `sda tty null random`

/lib / /lib64 — Библиотеки и модули ядра
Необходимые системные библиотеки и загружаемые модули ядра.
Примеры: `libc.so modules`

/media — Точки монтирования съёмных устройств
Автомонтирование USB, CD/DVD и др. временных носителей.
Примеры: `/media/usb`

/mnt — Временные точки монтирования
Используются для временного монтирования файловых систем вручную.
Примеры: `/mnt/disk`

/opt — Дополнительное ПО
Сторонние программы, не входящие в состав дистрибутива.
Примеры: `/opt/google`

/proc — Виртуальная файловая система процессов
Информация о процессах, ядре и системе в реальном времени.
Примеры: `/proc/cpuinfo`

/run — Данные о запущенной системе
PID-файлы, сокеты, временные данные сервисов и процессов.
Примеры: `/run/sshd.pid`

/sys — Информация о системе и устройствах
Интерфейс к ядру для получения сведений об устройствах и драйверах.
Примеры: `/sys/block`

КАТАЛОГИ ВНУТРИ /USR

`/usr` содержит большую часть пользовательских программ и данных.

- **/usr/bin** — Пользовательские команды
  Большинство обычных программ для пользователей.
- **/usr/sbin** — Системные утилиты
  Администрирование (необязательно для загрузки).
- **/usr/lib** — Библиотеки для программ в /usr/bin и /usr/sbin.
- **/usr/share** — Общие данные, документация, локализации, иконки.
- **/usr/local** — Локально установленные программы
  ПО, установленное вручную администраторами.
- **/usr/include** — Заголовочные файлы для разработки.
- **/usr/man** — Руководства (man-страницы).

ПОЛЕЗНО ЗНАТЬ

- Не изменяйте файлы в системных каталогах без необходимости.
- Храните пользовательские данные в `/home`.
- Логи и изменяемые данные — в `/var`.
- Устанавливайте стороннее ПО в `/opt` или `/usr/local`.
- Используйте `man <команда>` для справки.

ПРИМЕРЫ ПОЛНЫХ ПУТЕЙ

```
/etc/hosts              # файл настроек DNS
/var/log/syslog         # системный лог
/home/user/file.txt     # файл пользователя user
/usr/bin/python3        # исполняемый файл Python
/boot/vmlinuz           # ядро Linux
```

БЫСТРЫЕ КОМАНДЫ ДЛЯ РАБОТЫ С КАТАЛОГАМИ

```
ls /                    # посмотреть содержимое корня
ls /etc                 # список файлов в /etc
cd /var/log             # перейти в каталог
pwd                     # показать текущий путь
tree / -L 2             # дерево каталога (нужна утилита tree)
```

СОВЕТЫ АДМИНИСТРАТОРУ

- Регулярно проверяйте раздел `/var` на заполнение.
- Следите за логами в `/var/log`.
- Используйте `/tmp` только для временных файлов.
- Делайте резервные копии важных конфигов из `/etc`.
- Изучите структуру — это основа администрирования Linux!

ПАМЯТКА

Каталог	Назначение
/	Корень файловой системы
/etc	Конфигурационные файлы
/home	Личные данные пользователей
/var	Переменные данные и логи
/usr	Программы и общие данные
/boot	Файлы загрузки
/dev	Устройства

Понимание структуры каталогов — первый шаг к уверенной работе в Linux!

Практикуйтесь каждый день!

Подробный просмотр системного каталога

Задача: увидеть владельцев и права конфигурационных файлов.
Команда: **ls -la /etc**
Пример использования: **ls -la /etc**
Пример результата: строки с владельцем root и системными файлами.
Системная конфигурация обычно защищена от изменения обычным пользователем.

3. Файл и каталог в Linux

Файл — объект файловой системы, содержащий данные или представляющий системный объект.

Каталог — специальный файл, который связывает имена с объектами файловой системы.

Имя файла важно для человека, но система также использует inode и метаданные.

Для администратора файл — это не только содержимое, но и владелец, права, путь и тип.

СХЕМА: ФАЙЛ, КАТАЛОГ И INODE

В Linux всё есть файл. Каталоги — это специальные файлы.

Файловая система хранит данные в виде i-node (index node).
Имя файла в каталоге — это ссылка на inode.
Несколько имён могут указывать на один и тот же inode (жёсткие ссылки).

1. ЧТО ТАКОЕ?

ФАЙЛ

Набор данных (текст, программа, картинка и т.д.).
Имеет имя и хранится в inode.

КАТАЛОГ (ПАПКА)

Специальный файл, который содержит список имён и ссылки на inode других файлов и каталогов.

INODE (INDEX NODE)

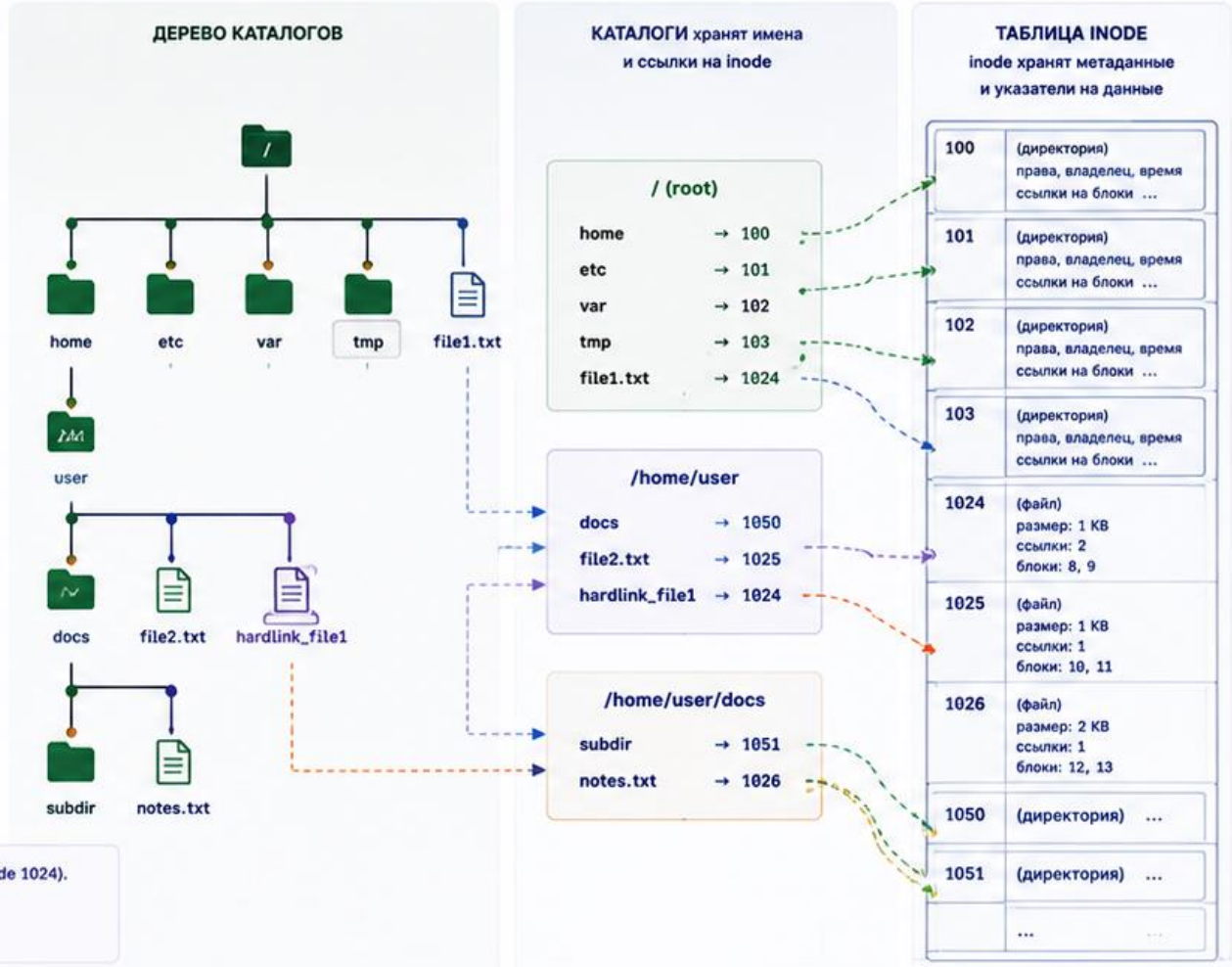
Структура данных в файловой системе, где хранится вся информация о файле: владелец, права, размер, время, кол-во ссылок и указатели на блоки с данными.

2. ПРИМЕР СВЯЗЕЙ

Имя	Тип	inode	Размер	Ссылки
file1.txt	файл	1024	1 KB	2
file2.txt	файл	1025	1 KB	1
hardlink_file1	файл (ссылка)	1024	1 KB	2
docs	каталог	1050	4 KB	2
subdir	каталог	1051	4 KB	2

`hardlink_file1` — это другое имя того же файла (inode 1024).
Удаление одного имени не удаляет данные, пока есть хотя бы одна ссылка.

3. КАК ЭТО УСТРОЕНО



4. ЧТО ВНУТРИ INODE?



5. ПОЛЕЗНЫЕ КОМАНДЫ

`ls -i`	показать inode файлов и каталогов
`stat <файл>`	подробная информация, включая inode
`find . -inum <N>`	найти файл по номеру inode
`ln <файл> <ссылка>`	создать жёсткую ссылку
`ls -l`	показать количество ссылок (столбец Links)
`df -i`	показать использование inode на файловой системе

6. КЛЮЧЕВЫЕ ИДЕИ

- Имя файла — это запись в каталоге, указывающая на inode.
- inode содержит всю информацию о файле, кроме его имени.
- Несколько имён (жёсткие ссылки) могут указывать на один inode.
- Файл удаляется из системы, когда счётчик ссылок = 0.
- Данные файла хранятся в блоках, inode содержит указатели на них.

Определение типа файла

Задача: не полагаться только на имя и расширение.

Команда: **file путь**

Пример использования: **file /etc/passwd**

Пример результата: **/etc/passwd: ASCII text**

Команда помогает понять фактический тип объекта.

4. Типы файлов Linux

Обычный файл хранит текст, данные или исполняемый код.

Каталог хранит соответствие имен и объектов.

Символическая ссылка указывает на другой путь.

Устройства, сокеты и pipe позволяют программам работать с оборудованием и обменом данными.

Зачем знать типы файлов

Нельзя одинаково обращаться с обычным файлом, каталогом и устройством.

Символическая ссылка может вести на несуществующий путь.

Службы используют сокеты и специальные файлы для взаимодействия.

Тип объекта влияет на права доступа и команды обслуживания.

СХЕМА: ТИПЫ ФАЙЛОВ LINUX

Linux всё — это файл. Тип файла определяет его назначение и способ работы с ним.

Каждый файл в Unix/Linux имеет тип.
Тип виден в первом символе вывода команды `ls -l`.
Всего в Linux 7 основных типов файлов.

1. КАК ОПРЕДЕЛИТЬ ТИП ФАЙЛА

Команда: `ls -l`

Первый символ в строке указывает тип файла:

```
$ ls -l
-rw-r--r--  1 user user  1024 May 20 10:00 file.txt
drwxr-xr-x  2 user user  4096 May 20 10:00 mydir
lrwxrwxrwx  1 user user    11 May 20 10:00 link -> file.txt
crw-rw---   1 root tty   4, 0 May 20 10:00 tty0
brw-r----   1 root disk  8, 0 May 20 10:00 sda
prw-rw----  1 user user     0 May 20 10:00 mypipe
srw-rw-rw-  1 user user     0 May 20 10:00 mysocket
```

Первый символ:

`-`	обычный файл
`d`	каталог (папка)
`l`	символическая ссылка
`c`	символьное устройство
`b`	блочное устройство
`p`	именованный канал (pipe)
`s`	сокет

2. ТИПЫ ФАЙЛОВ

1. Обычный файл

`-`

Файл с данными: текст, программа, изображение и т.д.

Не имеет специального формата для ядра.

Пример:
```
file.txt
script.sh
image.png
```

Первый символ: `-`

2. Каталог (папка)

`d`

Содержит имена других файлов и каталогов.

Каталог — это специальный файл.

Пример:
```
/home/user
/etc
/var/log
```

Первый символ: `d`

3. Символическая ссылка

`l`

Ссылка на другой файл или каталог.

Аналог ярлыка в Windows.

Пример:
```
link -> /etc/passwd
mydir_link -> /home/user
```

Первый символ: `l`

4. Символьное устройство

`c`

Устройства, к которым идёт побайтовый доступ (терминалы, порты и др.).

Пример:
```
/dev/tty0
/dev/null
/dev/random
```

Первый символ: `c`

5. Блочное устройство

`b`

Устройства, к которым идёт блочный доступ (диски, разделы и др.).

Пример:
```
/dev/sda
/dev/sda1
/dev/loop0
```

Первый символ: `b`

6. Именованный канал (pipe)

`p`

Канал для передачи данных между процессами.

Пример:
```
mypipe
```

Первый символ: `p`

7. Сокет

`s`

Используется для сетевого обмена данными между процессами.

Пример:
```
/var/run/docker.sock
mysocket
```

Первый символ: `s`

3. БИТЫ РЕЖИМА (ПРАВА ДОСТУПА)



Символы прав:

`r`	чтение (4)
`w`	запись (2)
`x`	выполнение (1)
`-`	нет права (0)

Пример для владельца (rw-)

```
r = 4
w = 2
- = 0
Итого: 6
```

4. ДОПОЛНИТЕЛЬНЫЕ БИТЫ

SUID (Set User ID)

При выполнении файла процесс получает права владельца файла.
Отображается как `s` или `S` в позиции `x` владельца.

Пример:
```
-rwsr-xr-x
```
(`s` в позиции владельца)

SGID (Set Group ID)

При выполнении файла процесс получает права группы файла.
Отображается как `s` или `S` в позиции `x` группы.

Пример:
```
-rwxr-sr-x
```
(`s` в позиции группы)

Sticky Bit (липкий бит)

Для каталогов: только владелец файла может удалять или переименовывать файлы в этом каталоге.
Отображается как `t` или `T` в позиции `x` остальных.

Пример:
```
drwxrwxrwt
```
(`t` в позиции остальных)

5. ПРИМЕРЫ ВЫВОДА `ls -l`

```
$ ls -l /etc/passwd /home /dev/null /dev/sda1 /tmp/mypipe /var/run/docker.sock
-rw-r--r--    1 root root    2345 May 20 10:00 /etc/passwd
drwxr-xr--    3 user user    4096 May 20 10:00 /home
crw-rw-rw-    1 root root     1,3 May 20 10:00 /dev/null
brw-r-----    1 root disk     8, 1 May 20 10:00 /dev/sda1
prw-r--r--    1 user user       0 May 20 10:00 /tmp/mypipe
srw-rw----    1 root docker     0 May 20 10:00 /var/run/docker.sock
```

Расшифровка:

`-`	обычный файл
`d`	каталог
`c`	символьное устройство
`b`	блочное устройство
`p`	именованный канал (pipe)
`s`	сокет

6. ПОЛЕЗНЫЕ КОМАНДЫ

`ls -l`	подробный список файлов
`file <имя>`	определить тип файла
`stat <имя>`	подробная информация (в т.ч. inode)
`find . -type f`	найти обычные файлы
`find . -type d`	найти каталоги
`find . -type l`	найти ссылки
`find . -type c`	найти символьные устройства
`find . -type b`	найти блочные устройства
`find . -type p`	найти именованные каналы
`find . -type s`	найти сокеты

Важно помнить

- Всё в Linux — это файл. Даже устройства, процессы и соединения.
- Тип файла влияет на то, как система с ним работает.
- Не изменяйте файлы устройств вручную — это может повредить систему.
- Практика — лучший способ понять файловую систему Linux!

Чтение типа по длинному списку

Задача: увидеть тип объекта по первому символу ls -l
Команда: **ls -l путь**
Пример использования: **ls -l /etc/passwd /tmp**
Пример результата: **-rw-r--r--** для файла, **drwxrwxrwt** для каталога.
Как читать результат: первый символ показывает тип: **-** -файл, **d** -каталог, **l** –ссылка.

5. Базовые операции: mkdir, touch, cp, mv, rm, rmdir

Базовые операции меняют структуру файловой системы.

mkdir создает каталог, touch создает пустой файл или меняет время.

cp копирует, mv перемещает или переименовывает

rm удаляет файлы, rmdir удаляет пустые каталоги.

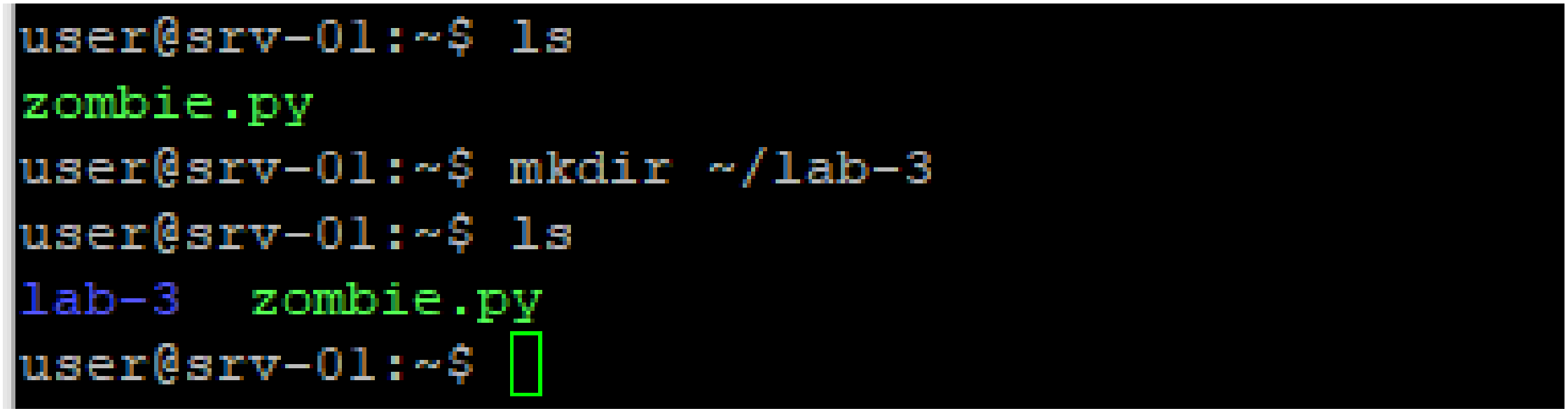
Любое изменение нужно выполнять в правильном каталоге и с пониманием результата.

Создание каталога

Команда: **mkdir -p путь**

Пример использования: **mkdir ~/lab3**

Пример результата: создан каталог lab3 внутри ~



Создание пустого файла

Команда: **touch файл**

Пример использования: **touch ~/lab-3/file1.txt**

Пример результата: появляется пустой файл file1.txt

touch не записывает содержимое, а создает файл или меняет время.

```
user@srv-01:~$ touch ./lab-3/laba-3.txt
user@srv-01:~$ ls ./lab-3/
laba-3.txt
user@srv-01:~$
```

Копирование

Команда: **cp**

Пример использования: **cp file1.txt file2.txt**

```
user@srv-01:~$ cp ./lab-3/laba-3.txt ./lab-3/laba-3-copy.txt
user@srv-01:~$ ls ./lab-3/
laba-3-copy.txt  laba-3.txt
user@srv-01:~$
```

Перемещение / переименование

Команда: **mv**

Пример использования: **mv file2.txt archive.txt**

```
laba-3-copy.txt  laba-3.txt
user@srv-01:~$ mv ./lab-3/laba-3-copy.txt ./lab-3/archive.txt
user@srv-01:~$ ls ./lab-3/
archive.txt  laba-3.txt
user@srv-01:~$
```

Удаление

Команды: **rm -i**

Пример использования: **rm -i archive.txt**

```
user@srv-01:~$ rm ./lab-3/archive.txt
user@srv-01:~$ ls ./lab-3/
laba-3.txt
user@srv-01:~$
```

БАЗОВЫЕ ОПЕРАЦИИ В LINUX

Работа с файлами и каталогами из командной строки

Все в Linux — это файлы и каталоги.
Эти команды помогут вам создавать, копировать, перемещать и удалять файлы и каталоги.

1. mkdir

Создать каталог

Создаёт новый каталог (папку).

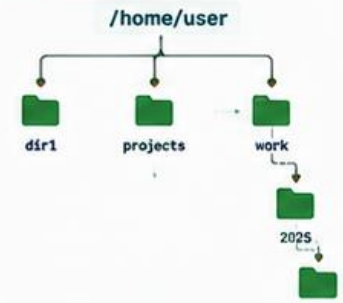
Синтаксис

```
mkdir [опции] имя_каталога
```

Примеры

```
$ mkdir dir1
$ mkdir projects
$ mkdir -p work/2025/june  # создать
      # вложенные каталоги
```

Что произойдёт



2. touch

Создать пустой файл или обновить время

Создаёт пустой файл. Если файл существует — обновляет изменения.

Синтаксис

```
touch [опции] имя_файла
```

Примеры

```
$ touch file.txt
$ touch notes.md
$ touch dir1/file.log
```

Что произойдёт



3. cp

Копировать файлы и каталоги

Копирует файлы и каталоги из одного места в другое.

Синтаксис

```
cp [опции] источник назначение
```

Примеры

```
$ cp file.txt file2.txt
$ cp notes.md /home/user/docs/
$ cp -r dir1 dir2     # копировать
                      # каталог целиком
```

Что произойдёт

Пример 1: файл



Пример 2: каталог



4. mv

Переместить или переименовать

Перемещает файл/каталог или переименовывает его.

Синтаксис

```
mv [опции] источник назначение
```

Примеры

```
$ mv file.txt docs/
$ mv oldname.txt newname.txt
$ mv dir1 dir_new  # переименовать
                   # каталог
```

Что произойдёт

Пример 1: перемещение



Пример 2: переименование



5. rm

Удалить файлы и каталоги

Удаляет файлы. С каталогами работает через опции.

Синтаксис

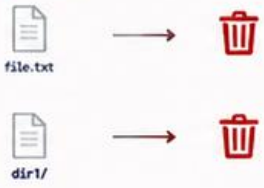
```
rm [опции] файл
```

Примеры

```
$ rm file.txt          # удалить файл
$ nm -i notes.md       # с запросом
$ rm -r dir1           # удалить
$ rm -rf dir2          # рекурсивно
                       # без подтверждения
```

Будьте осторожны!
Команда `rm -rf` удаляет безвозвратно.

Что произойдёт



6. rmdir

Удалить пустой каталоги

Удаляет ТОЛЬКО пустой каталог. Для непустых используйте `rm -r`.

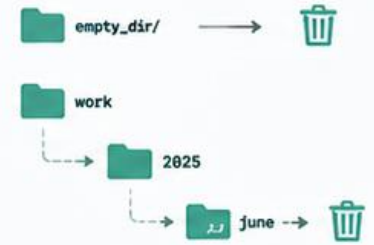
Синтаксис

```
rmdir [опции] каталог
```

Примеры

```
$ rmdir empty_dir
$ rmdir -p work/2025/june  # удалить
      # вложенные пустые каталоги
```

Что произойдёт



ПОЛЕЗНЫЕ ОПЦИИ

`-p`	создать родительские каталоги (`mkdir`, `rmdir`)	`-i`	интерактивный режим (`rm`)
`-r`	рекурсивно (для каталогов: `cp`, `rm`)	`-v`	подробный вывод
`-f`	принудительно, без подтверждений (`rm`, `mv`)	`-n`	не перезаписывать существующие файлы (`cp`, `mv`)

СОВЕТЫ

- Перед удалением всегда проверяйте команду.
- Используйте табуляцию для автодополнения имён.
- Команда `ls -l` поможет увидеть результат ваших действий.
- Читайте `man <команда>` для подробной справки.

СРАВНЕНИЕ КОМАНД

Команда	Что делает	Работает с	Пример
`mkdir`	Создаёт каталог	Каталоги	`mkdir mydir`
`touch`	Создаёт файл / обновляет время	Файлы	`touch file.txt`
`cp`	Копирует файлы/каталоги	Файлы и каталоги	`cp a.txt b.txt`
`mv`	Перемещает / переименовывает	Файлы и каталоги	`mv old.txt new.txt`
`rm`	Удаляет файлы/каталоги	Файлы и каталоги	`rm -r mydir`
`rmdir`	Удаляет пустой каталог	Только пустые каталоги	`rmdir empty_dir`

Практика — лучший способ освоить Linux!

Начните с малого, экспериментируйте и не бойтесь ошибок!

Команда echo в командной строке Linux Bash

echo — это одна из самых часто используемых встроенных команд в оболочке Bash. Её основное назначение — выводить строку текста (или значения переменных) в стандартный вывод (stdout), то есть прямо на экран терминала.

Простая аналогия: Команда echo работает как сетевой или системный «попугай» — она просто громко повторяет в терминал всё, что вы передадите ей в качестве аргумента. Также она является аналогом функции print() в языках программирования.

Зачем она нужна администратору? (Основные сценарии)

- Вывод информационных сообщений: Используется в автоматических bash-скриптах, чтобы сообщать администратору о текущем статусе выполнения (например: «Бэкап успешно создан»).
- Просмотр системных переменных окружения: Позволяет узнать текущие настройки пользователя или пути к программам (например, посмотреть, под каким пользователем мы работаем или где ищутся исполняемые файлы).
- Быстрая запись текста в файлы (в связке с перенаправлением **>** и **>>**): Вместо открытия тяжелых текстовых редакторов типа nano или vim, администратор может в одну строчку создать файл или дописать параметр в конфигурацию.

Синтаксис и примеры использования

Базовый синтаксис выглядит очень просто:

echo [опции] [строка_текста или переменная]

А) Простой вывод текста

Текст можно писать как в кавычках (одинарных или двойных), так и без них, если в нем нет спецсимволов.

```
echo Привет, сисадмин!
echo "Hello, Linux World"
```

B) Вывод переменных окружения

Чтобы echo вывела не имя переменной, а её содержимое, перед именем обязательно ставится знак доллара $:

```
echo $USER    # Выведет имя текущего пользователя (например: root или viktor)
echo $HOME    # Выведет абсолютный путь к домашнему каталогу (например: /home/viktor)
echo $PATH    # Выведет список каталогов, где система ищет исполняемые команды
```

C) Использование ключевых опций (флагов)
По умолчанию **echo** всегда автоматически добавляет перевод строки в самом конце своего вывода. Это поведение можно изменить:
Опция **-n** — отменяет автоматический перевод строки в конце вывода. Следующий вывод терминала останется на той же строке.

```
echo -n "Текст без перевода строки"
```

Опция **-e** — включает поддержку escape-последовательностей (специальных невидимых символов, начинающихся с обратного слэша ****).
\n — принудительный перевод строки (перенос текста на новую строчку).
\t — горизонтальная табуляция (отступ в 4-8 пробелов).

```
# Текст разделится на две строки прямо посреди вывода
echo -e "Первая строка\nВторая строка"

# Выведет текст со сдвигом (табуляцией)
echo -e "Параметр:\tЗначение"
```

D) Создание файлов без редактора.
В связке с операторами перенаправления потоков (из темы базовой работы с ФС) **echo** превращается в инструмент быстрого создания конфигураций:
Символ **>** (Перезапись / Создание): Создает новый файл с указанным текстом. Если файл уже существовал, его старое содержимое полностью удалится.

```
echo "nameserver 8.8.8.8" > test_resolv.conf
```

```
user@srv-01:~$ ping 8.8.8.8 -c 3 > ping.txt
user@srv-01:~$ cat ping.txt
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=108 time=98.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=108 time=100 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=108 time=99.7 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 98.932/99.648/100.312/0.564 ms
```

Символ >> (Дозапись в конец): Берет текст и аккуратно дописывает его в самый конец существующего файла, сохраняя старые данные.

```
echo "nameserver 1.1.1.1" >> test_resolv.conf
```

КОМАНДА echo в командной строке Linux Bash

Вывод текста и переменных на экран или в файл

echo — одна из самых простых и часто используемых команд в Bash.
Используется для вывода строк, переменных и управления выводом.

1. ЧТО ДЕЛАЕТ echo

Команда echo отображает (печатает) текст, значения переменных и специальные символы в стандартный вывод (экран) или перенаправляет их в файл.



2. БАЗОВЫЕ ПРИМЕРЫ

```
$ echo Привет, мир!
Привет, мир!
$ echo 'Добро пожаловать в Linux'
Добро пожаловать в Linux
$ echo "Linux — это интересно!"
Linux — это интересно!
$ echo
(пустая строка)
$ echo -n "Без перевода строки..."
Без перевода строки... (курсор остаётся на этой же строке)
```

3. ПЕРЕМЕННЫЕ С echo

```
$ name="Alice"
$ echo $name                          Alice
$ echo "Привет, $name!"               Привет, Alice!
$ echo 'Привет, $name!'               Привет, $name!
$ echo "Сегодня: $(date +%d.%m.%Y)"   Сегодня: 24.05.2025
```

4. СПЕЦИАЛЬНЫЕ СИМВОЛЫ (ЭКРАНИРОВАНИЕ)

Используйте -e для интерпретации специальных последовательностей.

Последовательность	Что означает	Пример команды	Результат
\n	перевод строки	echo -e "Строка 1\nСтрока 2"	Строка 1 Строка 2
\t	табуляция (гориз. отступ)	echo -e "Имя:\tAlice"	Имя: Alice
\r	возврат каретки	echo -e "Привет\rМир"	Мир
\	обратный слэш	echo -e "Путь: C:\\Linux"	Путь: C:\Linux
"	двойные кавычки	echo -e 'Он сказал: "Привет"'	Он сказал: "Привет"
\0NNN	символ по ASCII (восьм.)	echo -e "Символ: \101"	Символ: A

Примеры с -e:

```
$ echo -e "Line 1\nLine 2\nLine 3"              # вывод в три строки
$ echo -e "Колонка 1\tКолонка 2\tКолонка 3"     # табуляция
$ echo -e "Отсчёт: 3\r2\r1\rСтарт!"             # эффект перезаписи одной строки
$ echo -e "Папка: /home/user\nФайл: file.txt"   # перевод строки и текст
```

5. ПЕРЕНАПРАВЛЕНИЕ В ФАЙЛ

```
$ echo "Привет" > file.txt                      # создать/перезаписать файл
$ echo "Вторая строка" >> file.txt              # добавить в конец файла
$ echo -e "Строка 1\nСтрока 2" > file.txt       # записать с переводами строк
$ cat file.txt                                  # вывести содержимое файла
```

Содержимое file.txt:

```
Привет
Вторая строка
Строка 1
Строка 2
```

6. ПОЛЕЗНЫЕ ОПЦИИ

Опция	Описание
-n	Не выводить символ новой строки в конце. Пример: echo -n "Текст"
-e	Включить интерпретацию специальных символов \n \t \r и др. По умолчанию может не работать в некоторых оболочках.
-E	Отключить интерпретацию специальных символов (по умолчанию в Bash).
--help	Показать справку.

7. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

```
$ echo "=== Системная информация ==="                  # заголовок
$ echo -e "Пользователь:  \t$(whoami)"                 # имя пользователя
$ echo -e "Дата:          $(date +%d.%m.%Y)"           # дата
$ echo -e "Оболочка:      $SHELL"                      # текущая оболочка
$ echo -e "Домашняя папка: \t$HOME"                    # переменная среды
```

8. СОВЕТЫ

- Для многострочного текста используйте echo -e или cat <<EOF ... EOF
- Используйте одинарные кавычки, чтобы избежать подстановки переменных.
- Используйте двойные кавычки, если нужны переменные и спецсимволы.
- echo удобно применять в скриптах для вывода сообщений пользователю.

КРАТКО

echo [опции] [строка или $переменная]

Простой вывод

```
$ echo "Привет"
Привет
```

Без новой строки

```
$ echo -n "Текст"
Текст (курсор здесь)
```

С переводом строки

```
$ echo -e "A\nB"
A
B
```

С табуляцией

```
$ echo -e "A\tB"
A       B
```

В файл (перезапись)

```
$ echo "Данные" > file.txt
```

В файл (добавление)

```
$ echo "Ещё строка" >> file.txt
```

ВАЖНО

echo — просто, но мощно!
Маленькая команда, без которой не обойтись в Bash.

6. Поиск файлов: find, locate на уровне назначения

Поиск нужен, когда неизвестно точное расположение файла.

find обходит дерево каталогов и ищет актуальное состояние файловой системы.

locate ищет быстрее, но по заранее подготовленной базе имен.

Поиск файла в рабочем каталоге

Задача: найти файлы по имени или шаблону.

Команда: **find каталог -name шаблон**

Пример использования: **find ~/lab3 -name "*.txt"**

Пример результата: список найденных txt-файлов.

```
user@srv-01:~$ find ./lab-3 -name *.txt
./lab-3/archive.txt
./lab-3/test.txt
./lab-3/laba-3.txt
user@srv-01:~$ find ./lab-3 -name *.txt | grep "laba"
./lab-3/laba-3.txt
user@srv-01:~$
```

Быстрый поиск по базе

Задача: найти известный файл по части имени.
Команда: **locate имя**
Пример использования: **locate sshd_config**
Пример результата: пути к файлам с таким именем.
Как читать результат: если база locate устарела, только что созданные файлы могут не найтись.

```
user@srv-01:~$ sudo updatedb
user@srv-01:~$ locate interfaces
/etc/network/interfaces
/etc/network/interfaces.d
/usr/share/doc/ifupdown/examples/generate-interfaces.pl
/usr/share/doc/ifupdown/examples/network-interfaces
/usr/share/man/man5/interfaces.5.gz
user@srv-01:~$
```

ПОИСК ФАЙЛОВ: find, locate

Два мощных инструмента для поиска файлов в Linux

find и locate позволяют быстро находить файлы и каталоги по имени и другим параметрам.
find ищет в реальном времени, locate использует индекс (базу данных).

find — поиск в реальном времени

find обходит файловую систему начиная с указанного каталога и выводит пути, удовлетворяющие заданным условиям.

```
find [путь] [условия] [действия]
```

- Всегда актуальные результаты
- Поддержка сложных условий
- Может быть медленным на по-больших дисках

ОСНОВНЫЕ ПРИМЕРЫ

```
$ find /home/user -name file.txt

$ find /var/log -type f -name "*.log" #
                # найти файлы с расширением .log
$ find . -type d -name "docs"
                # найти каталоги с именем docs
$ find / -size +100M
                # файлы больше 100 МБ
$ find /tmp -mtime -7
                # изменённые за последние 7 дней
$ find / -user user1
                # файлы, принадлежащие user1
$ find . -name "*.py" -exec grep
                -l "TODO" {}
# найти .py файлы, содержащие "TODO"
```

ПОЛЕЗНЫЕ ПАРАМЕТРЫ find

Параметр	Что делает (примеры)
-name "имя"	по имени (поддерживает шаблоны) -name "*.txt"
-iname "имя"	по имени (без учёта регистра) -iname "Readme.md"
-type f / d / l	тип: файл (f), каталог (d), ссылка (l) -type f
-size +N / -N	размер: больше (+N) или меньше (-N) -size +100M, -size -1M
-mtime -N / +N	изменённый: за N дней назад / более N -mtime -7
-user имя	принадлежит пользователю
-perm режим	по правам доступа -perm 755
-exec команда {} \;	выполнить команду для найденных -exec rm {} \;
-print	вывести путь (по умолчанию)

ПРИМЕРЫ СЛОЖНЫХ УСЛОВИЙ

```
$ find /home/user -type f \( -name "*.txt" -o -name "*.md" \)   # файлы .txt или .md
$ find /var -type f -size +50M -mtime -30                       # файлы >50МБ, изменённые за 30 дней
$ find / -type f -perm -4000                                    # файлы с установленным SUID битом
$ find / -path "*/cache/*" -prune -o -type f -print             # исключить каталоги cache
$ find /home -name "*.bak" -delete                              # найти и удалить .bak файлы
```

ПОДСКАЗКИ

Начинайте поиск с конкретного каталога, а не с / — это ускорит работу.
Используйте -print (по умолчанию) для просмотра результатов.

СРАВНЕНИЕ: find vs locate

	find	locate
Принцип работы	Обходит файловую систему в реальном времени	Использует индексную базу данных (mlocate)
Скорость	Медленнее (зависит от размера диска)	Очень быстро
Актуальность	Всегда актуальные данные	Может быть неактуально (нужна обновление БД)
Гибкость	Очень гибкий: условия, размер, дата, права...	Только по имени файла
Поиск	С учётом регистра (по умолчанию)	Без учёта регистра
Где искать	Указывает пользователь (каталог или вся система)	Вся система (по индексу)
Обновление	Не требуется	Требуется обновлять базу: updatedb

КАК РАБОТАЕТ locate

1. updatedb создаёт индекс → 2. locate ищет в индексе → 3. возвращает результаты

По умолчанию индекс хранится в: /var/lib/mlocate/mlocate.db

locate — быстрый поиск по индексу

locate ищет файлы по имени в заранее созданной базе данных.

```
locate [имя_файла]
```

- Очень быстрый поиск
- Простой синтаксис
- Требует обновления базы (не всегда актуален)

ОСНОВНЫЕ ПРИМЕРЫ

```
$ locate file.txt
                # найти file.txt
$ locate '*.log'
                # найти файлы с расширением .log
$ locate config
                # найти все файлы с config в имени
$ locate -i readme
                # поиск без учёта регистра
$ locate -r '\.conf$'
                # по регулярному выражению
$ locate -n 10 ssh
                # показать первые 10 результатов
$ locate -c passwd
                # показать количество совпадений
```

ПОЛЕЗНЫЕ ПАРАМЕТРЫ locate

Параметр	Что делает (примеры)	
-i	без учёта регистра	locate -i file
-r	по регулярному выражению	locate -r "\.conf$"
-n N	показать первые N результатов	locate -n 20 file
-l N	показать последние N результатов	locate -l 20 file
-c	показать количество совпадений	locate -c file
-d N	искать в базе N дней назад	locate -d 3 file
--regex	включить регулярные выражения	locate --regex '.*\.log$'

ОБНОВЛЕНИЕ БАЗЫ locate (mlocate)

Для актуальности результатов нужно регулярно обновлять индекс:

```
$ sudo updatedb                    # обновить базу (нужны права root)
$ sudo updatedb -U /home/user      # обновить только указанный путь
$ sudo updatedb -o /tmp/mlocate.db # указать другой файл БД
```

АВТОМАТИЧЕСКОЕ ОБНОВЛЕНИЕ

На большинстве систем updatedb запускается автоматически по расписанию (cron/systemd timer).

Проверьте: `systemctl list-timers | grep mlocate`

КОГДА ЧТО ИСПОЛЬЗОВАТЬ?

Используйте find, когда нужно:

- точные и актуальные результаты
- поиск по содержимому, дате, размеру, правам
- сложные условия и действия (удаление, копирование и т.д.)

Используйте locate, когда нужно:

- быстрый поиск по имени файла
- не важна точность до секунды
- простой поиск без сложных условий

Совет: объединяйте команды! Например: `locate '*.log' | xargs grep 'ERROR'`

Практика — лучший способ освоить find и locate!

7. inode: назначение и связь с именем файла

inode — служебная запись файловой системы с метаданными объекта.

В inode хранится владелец, права, размер, времена и ссылки на блоки данных.

Имя файла хранится в каталоге и указывает на inode.

Это объясняет, почему у одного файла может быть несколько имен через hard link.

СХЕМА: INODE И ЖЁСТКАЯ ССЫЛКА

В Linux несколько имён (ссылок) могут указывать на один и тот же inode.

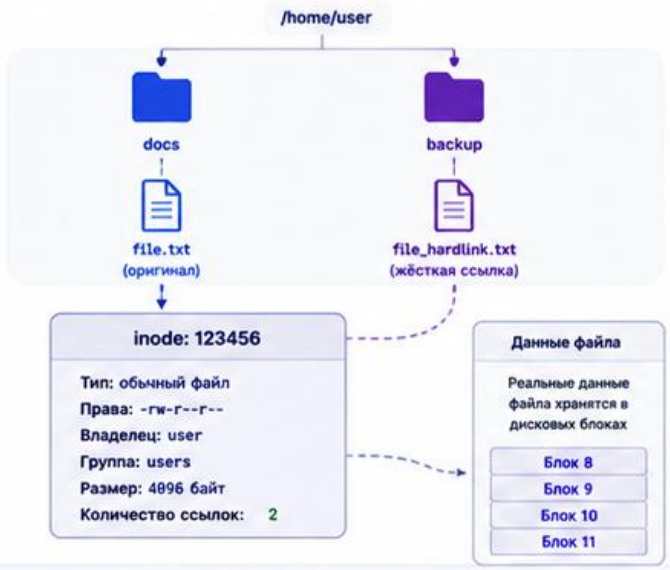
> inode — это индексный дескриптор (метаданные файла + указатели на данные).
> Жёсткая ссылка — это другое имя для того же inode в той же файловой системе.

1. ЧТО ТАКОЕ INODE?

Каждый файл и каталог в Linux имеет inode.
В inode хранится вся информация о файле, кроме его имени.



2. ЖЁСТКАЯ ССЫЛКА: КАК ЭТО РАБОТАЕТ



> Обе ссылки (file.txt и file_hardlink.txt) указывают на один и тот же inode.
> Поле «Количество ссылок» показывает, сколько имён ссылаются на этот inode.

3. ЧТО ПРОИСХОДИТ ПРИ ДЕЙСТВИЯХ

Действие	Команда	Что происходит	Результат
Создать жёсткую ссылку	`ln file.txt file_hardlink.txt`	Создаётся новое имя, ссылающееся на тот же inode.	Количество ссылок: 2
Удалить одну ссылку	`rm file.txt`	Удаляется только имя file.txt. Inode и данные остаются.	Количество ссылок: 1
Удалить последнюю ссылку	`rm file_hardlink.txt`	Удаляется последнее имя. Количество ссылок становится 0. Inode и данные удаляются.	Файл и данные удалены

4. КАК ПОСМОТРЕТЬ INODE И ССЫЛКИ

```
$ ls -li
 inode  links  права       владелец  группа  размер  дата и время  имя
123456  2      -rw-r--r--  user      users   4096    May 20 10:00  file.txt
123456  2      -rw-r--r--  user      users   4096    May 20 10:00  file_hardlink.txt

$ stat file.txt
  File: file.txt
  Inode: 123456           Links: 2
  ...
```

Inode — номер inode
Links — количество жёстких ссылок

5. ВАЖНЫЕ ОСОБЕННОСТИ

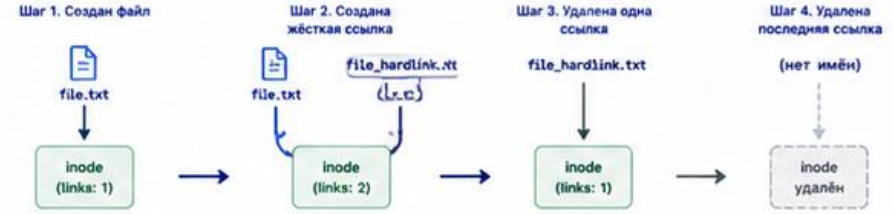
- Жёсткая ссылка может быть создана только внутри одной файловой системы.
- Нельзя создать жёсткую ссылку на каталог (обычно).
- Жёсткая ссылка не увеличивает размер файла на диске.
- Все жёсткие ссылки равноправны. Нет «главной» ссылки.
- Права и владельцы у всех ссылок могут быть разными.
- Пока количество ссылок > 0, данные файла не удаляются.

> **Не путайте с символической ссылкой (soft link)!**
> Символическая ссылка — это отдельный файл, который указывает на путь.

6. ПРИМЕР НА ПРАКТИКЕ

```
$ echo "Привет, Linux!" > file.txt          # создаём файл
$ ls -li file.txt
123456  1 -rw-r--r--  user  users  15 May 20 10:00 file.txt
$ ln file.txt file_hardlink.txt             # создаём жёсткую ссылку
$ ls -li
123456  2 -rw-r--r--  user  users  15 May 20 10:00 file.txt
123456  2 -rw-r--r--  user  users  15 May 20 10:00 file_hardlink.txt
$ rm file.txt                               # удаляем одну
$ ls -li
123456  1 -rw-r--r--  user  users  15 May 20 10:00 file_hardlink.txt
$ rm file_hardlink.txt        # удаляем последнюю ссылку
$ ls -li
# файл исчез — inode и данные удалены
```

7. ВИЗУАЛЬНОЕ ПРЕДСТАВЛЕНИЕ ЖИЗНЕННОГО ЦИКЛА



Главное: inode — это «паспорт» файла в файловой системе.

Жёсткая ссылка — это дополнительное имя для того же inode.

Количество ссылок показывает, сколько имён ссылается на inode.

Данные удаляются только тогда, когда количество ссылок становится 0.

Команды: `ln` — создать жёсткую ссылку
`ls -li`, `stat` — посмотреть inode и ссылки

Просмотр inode и числа ссылок

Задача: увидеть метаданные объекта и количество имен.

Команда: **stat файл**

Пример использования: **stat file1.txt**

Пример результата: Inode: 123456; Links: 1

Links показывает, сколько жестких ссылок указывает на этот inode.

8. Жесткие ссылки и символические ссылки

Жесткая ссылка — второе имя для того же inode
Символическая ссылка — отдельный файл, в котором записан путь к цели
Hard link продолжает работать после удаления одного имени
Symlink может стать битой ссылкой, если целевой путь исчез

СХЕМА: HARD LINK И SYMLINK

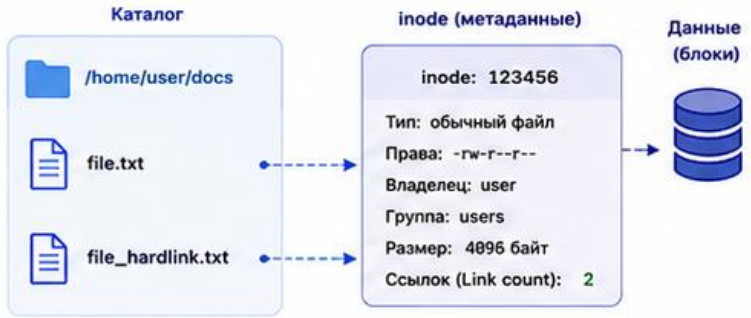
Два типа ссылок в Linux: принцип работы и отличия

Файл на диске — это inode (метаданные) + данные.
Ссылки — это имена, которые указывают на inode или на другой путь.

1. HARD LINK — жёсткая ссылка

Жёсткая ссылка — это другое имя для того же inode.

Указывает напрямую на inode



Как это работает

- Обе ссылки (имена) указывают на один и тот же inode.
- Данные файла хранится один раз.
- Link count показывает количество жёстких ссылок.

Особенности

- Нельзя создать на каталог.
- Работает только внутри одной файловой системы.
- Если удалить одну ссылку — файл останется, пока link count > 0.

Пример

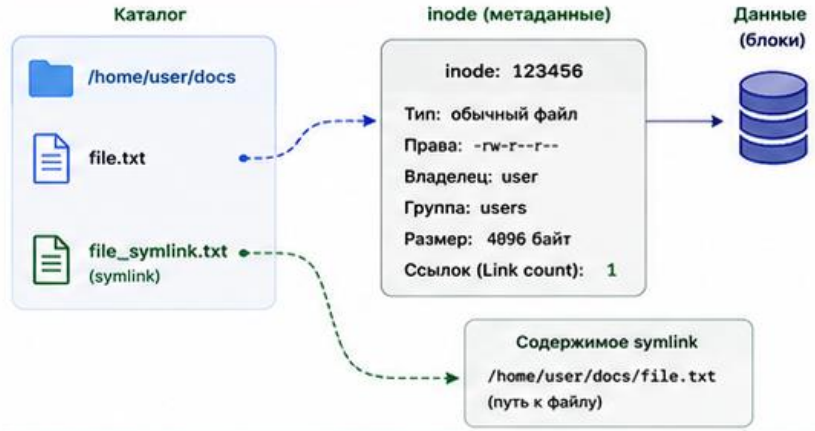
```
$ ln file.txt file_hardlink.txt       # создать жёсткую ссылку
$ ls -li
123456   2 -rw-r--r--    user    users   4096 May 20 10:00 file.txt
123456   2 -rw-r--r--    user    users   4096 May 20 10:00 file_hardlink.txt

$ rm file.txt                         # удалить одну ссылку
$ ls -li
123456   1 -rw-r--r--    user    users   4096 May 20 10:00 file_hardlink.txt
```

2. SYMLINK — символическая ссылка

Символическая ссылка — это специальный файл, который хранит путь к другому файлу или каталогу.

Указывает на путь (не на inode)



Как это работает

- Symlink — это отдельный файл, в котором хранится путь к цели.
- Имеет свой собственный inode.
- Если цель перемещена или удалена — ссылка «сломается» (broken link).

Особенности

- Можно создавать на файлы и каталоги.
- Может указывать на объекты в других файловых системах.
- Не увеличивает link count исходного файла.

Пример

```
$ ln -s file.txt file_symlink.txt     # создать символическую ссылку
$ ls -li
123456   1 -rw-r--r--  user   users  4096 May 20 10:00  file.txt
123789   1 lrwxrwxrwx  user   users    24 May 20 10:00  file_symlink.txt  ->  file.txt

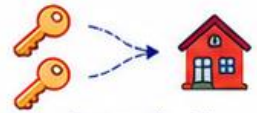
$ rm file.txt                         # удалить оригинал
$ ls -l
lrwxrwxrwx   1 user    users    24 May 20 10:00  file_symlink.txt  ->  file.txt (broken)
```

3. СРАВНЕНИЕ HARD LINK И SYMLINK

Характеристика	Hard Link (жёсткая)	Symlink (символическая)
Что указывает	на inode	на путь
Тип	не отдельный файл	отдельный файл (с путём)
link count	увеличивает	не влияет
Можно на каталог	нет	да
Работает между ФС	нет	да
При удалении оригинала	файл остаётся (если link count > 0)	ссылка становится «битой»
Производительность	быстрее	чуть медленнее
Команда создания	`ln`	`ln -s`

4. ВИЗУАЛЬНАЯ АНАЛОГИЯ

Hard link — как дубликат ключа



Один дом (inode), несколько ключей (имён)

Symlink — как указатель (ярлык)



Указатель ведёт к дому (пути), если дом уберут — указатель бесполезен

5. ПОЛЕЗНЫЕ КОМАНДЫ

`ln источник цель`	создать жёсткую ссылку
`ln -s источник цель`	создать символическую ссылку
`ls -li`	показать inode и link count
`readlink файл`	показать путь в symlink
`stat файл`	подробная информация (inode и тип)

6. ПРОВЕРИТЬ ТИП ФАЙЛА

```
$ file file_hardlink.txt
file_hardlink.txt: ASCII text
$ file file_symlink.txt
file_symlink.txt: symbolic link to file.txt
```

`ls -l` обозначения

- `-` — обычный файл
- `l` — символическая ссылка

Важно помнить

- Hard link = другое имя для того же inode.
- Symlink = файл, который хранит путь.
- Hard link нельзя создать на каталог и между разными ФС.
- Symlink удобен для ярлыков, ссылок на каталоги и внешние диски.
- Используйте `ls -li`, чтобы увидеть inode и количество ссылок (link count).

Создание hard link и symlink

Команда: **ln / ln -s**

Пример использования:

ln file1.txt hard1.txt

ln -s file1.txt sym1.txt

результат: hard1.txt имеет тот же inode, sym1.txt указывает на путь.

hard link связан с inode, symlink связан с именем пути

9. Типовые ошибки при удалении, копировании и работе со ссылками

- Удаление файла не из того каталога из-за непроверенного pwd;
- Использование rm -r без просмотра содержимого каталога;
- Ожидание, что symlink сохранит данные после удаления цели;
- Путаница между копией и hard link;
- Копирование ссылок без понимания, копируется ссылка или цель;

Итоги лекции

- Linux использует единое дерево каталогов от корня **/**;
- Основные каталоги имеют устойчивое назначение для администрирования;
- Файл в Linux — это содержимое, тип, права, владелец и inode;
- Hard link и symlink решают разные задачи;
- Безопасная работа начинается с проверки пути, типа объекта и ожидаемого результата;

Контрольные вопросы

1. Чем дерево каталогов Linux отличается от букв дисков Windows?
2. Для чего нужны /etc, /var и /home?
3. Что такое каталог с точки зрения файловой системы?
4. Что показывает inode?
5. Чем копия отличается от hard link?
6. Почему symlink может стать битой ссылкой?
7. Когда использовать find, а когда locate?
8. Какие проверки нужно сделать перед rm -r?